

Using the Reproduction Set repro/

Scripts and data of the dynamic inoperability (DIIM) analysis of Spain's agri-food system
submitted to the
International Journal of Critical Infrastructure Protection

Package `envio_IJCIP`, directory `repro/`

3 October 2026

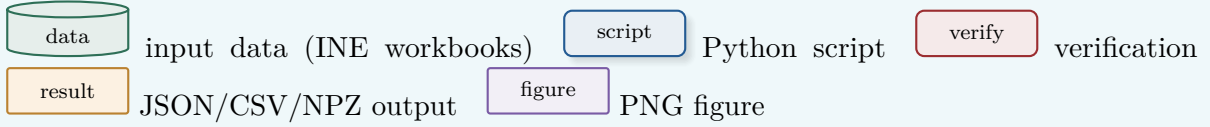
Abstract

This guide explains how to use the reproduction set shipped with the manuscript *Quantifying Cyber-Induced Cascades under Climate-Conditioned Recovery to Prioritise Critical-Infrastructure Defence: A Dynamic Inoperability Analysis of Spain's Agri-Food System*. The directory holds two official INE workbooks, twenty Python scripts and the JSON, CSV and NPZ files they produce. One command, `python3 run_all.py`, regenerates every number, table and figure of the manuscript in about fifty seconds and fails on the first discrepancy. The guide describes the environment, the order of the sixteen pipeline stages, what each script reads and writes, the values that identify a faithful run, and the scope of the Censys exposure material that is *not* part of the pipeline. Every statement was checked by running the set on a clean copy on 3 October 2026 with Python 3.12.3; the run regenerated all committed result files byte for byte.

Contents

1	What the directory contains	2
2	Environment	2
3	The pipeline at a glance	3
3.1	Run times	3
3.2	Running a single stage	4
4	The model the scripts implement	4
4.1	From 65 INE products to 8 sectors	5
4.2	Scenario catalogue	6
5	Script by script	6
6	What a correct run looks like	8
7	Data provenance	10
8	The Censys exposure material	10
9	The six figures	11
10	Before editing anything	13

i Colour code used throughout this guide



1 What the directory contains



Figure 1: Layout of `repro/`. Blue items are scripts, red the verification stage, amber the committed results, green the input data, purple the figure directory outside the clone.

2 Environment

The pipeline needs Python 3 with four third-party packages. $\text{\TeX}$ is not needed to run the scripts; it is needed only to compile the manuscript and this guide.

Component	Reference run	Minimum requirements.txt	in	Role
Python	3.12.3 (Linux)	3		interpreter
numpy	1.26.4	≥ 1.24		linear algebra, random streams (PCG64)
scipy	1.11.4	≥ 1.10		matrix exponential, ODE check, distributions
matplotlib	3.6.3	≥ 3.6		figures (stage 16 only)
openpyxl	3.1.5	≥ 3.1		reading the INE workbooks (stages 1–4 only)

>_ Set up and run

```

cd repro
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python3 run_all.py          # about 50 s; ends with "PIPELINE COMPLETED in 50 s"
  
```

i Reproducibility of the random draws

Point values do not depend on package versions beyond floating-point noise. Monte Carlo percentiles depend on numpy's `default_rng` (PCG64) stream seeded with 20260719, with child seeds [20260719, *k*] for the two-phase scenario A1 and for the extra seeds of the uncertainty decomposition. The stream is stable across the versions above, so envelopes reproduce bit for bit.

3 The pipeline at a glance

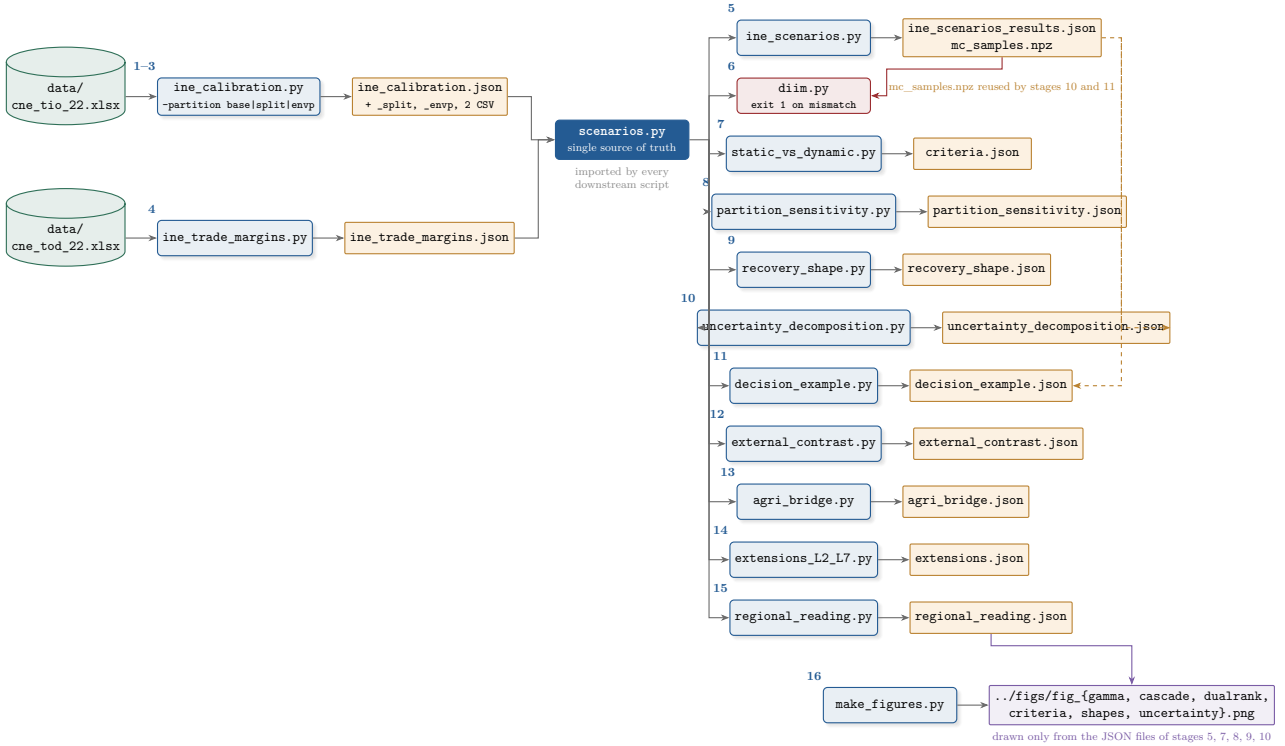


Figure 2: Data flow of `run_all.py`. Stage numbers are the positions in the driver's list. Solid black arrows are read dependencies; the red arrow is the comparison performed by the verification stage; dashed amber arrows mark the reuse of the stored structural Monte Carlo draws.

`run_all.py` executes the sixteen stages of Figure 2 in order, each as a separate Python process with the directory of the script as working directory. If any stage exits with a non-zero code, the driver prints `PIPELINE FAILED at <script>` and stops. Only the first two scripts touch the INE workbooks. Every other script imports `scenarios.py` and reads the JSON files written upstream, so the manuscript, the stored results and the figures cannot drift apart.

3.1 Run times

Figure 3 shows the wall-clock time of each stage measured on the verification run of 3 October 2026 (total 50 s). Two stages account for nearly all the time: the Monte Carlo of `ine_scenarios.py` and the uncertainty decomposition. Everything else runs in a second or less.

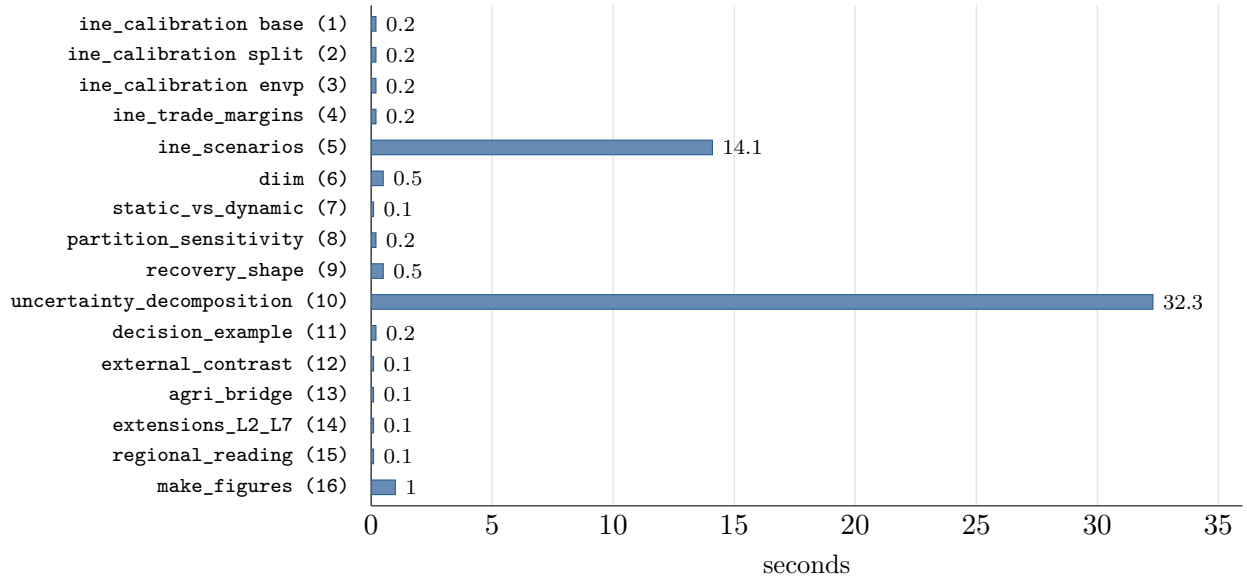


Figure 3: Wall-clock time per stage (Python 3.12.3, pinned package versions). Stage numbers in parentheses.

3.2 Running a single stage

Each script is self-contained and resolves its paths relative to its own location, so it can be launched from any working directory. Stages must nevertheless respect the dependency order of Figure 2: a script fails with a missing-file error if the JSON it needs has not been written. The three calibration variants are selected by option:

>_ Individual stages

```
python3 repro/ine_calibration.py --partition base      # also: split, envp
python3 repro/ine_scenarios.py                       # rewrites results JSON and
mc_samples.npz
python3 repro/diim.py; echo "exit code $?"           # 0 = stored baseline reproduced
python3 repro/make_figures.py                        # ../figs/*.png from the JSON files
```

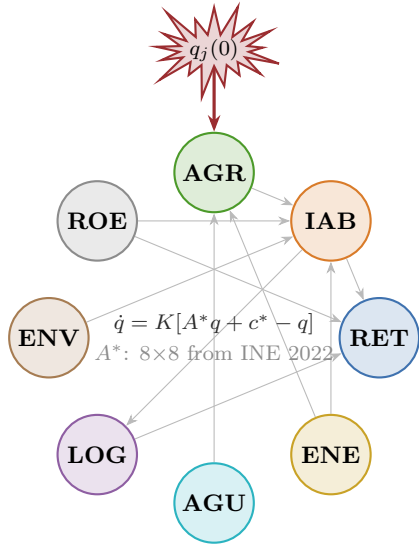
⚠ The -audit option

`run_all.py -audit` additionally runs `../audit.py`, a manuscript audit that compares the JSON results with the numbers typed in the `.tex` sources. That script is not included in this `envio_IJCIP` package, so the option fails here with a file-not-found error. Use the plain `python3 run_all.py`.

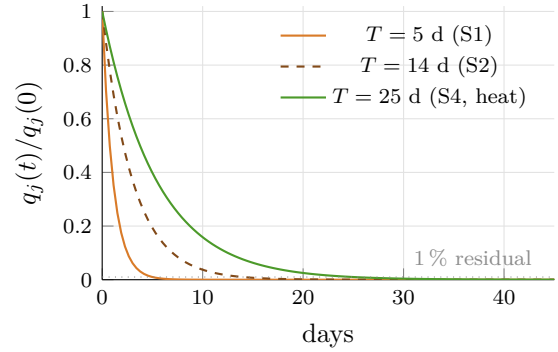
4 The model the scripts implement

The code implements the demand-side dynamic inoperability input–output model (DIIM) of Santos and Haines with the recovery calibration of Lian and Haines. All definitions live in `scenarios.py`; the formulas below are transcribed from its docstring.

Interdependence	$a_{ij}^* = Z_{ij}/x_i$: share of supplier i 's sales absorbed by customer j . $A^* = D^{-1}AD$ with $D = \text{diag}(x)$.
Recovery	$k_i = \ln 100 / (T_i(1 - a_{ii}^*))$: an isolated sector keeps 1 % of its inoperability after T_i days.
Dynamics	$\dot{q} = K[A^*q + c^* - q]$, with $K = \text{diag}(k_i)$ and c^* the sustained forcing (zero for transient shocks).
Loss	$\Gamma(H) = \frac{1}{365} x^\top \int_0^H q(t) dt$ in EUR million at 2022 basic prices, reported at $H = 120$ days.
Closed form	$\Gamma_\infty = \sum_j \text{targeted } q_j(0) T_j (1 - a_{jj}^*) w_j / (365 \ln 100)$, with exposure weight $w_j = [x^\top (I - A^*)^{-1}]_j = x_j m_j$ and m_j the Leontief output multiplier.



(a) A cyber shock sets the initial inoperability $q_j(0)$ of a targeted sector; A^* propagates it to customers and suppliers. Grey arrows are illustrative couplings, not weighted.

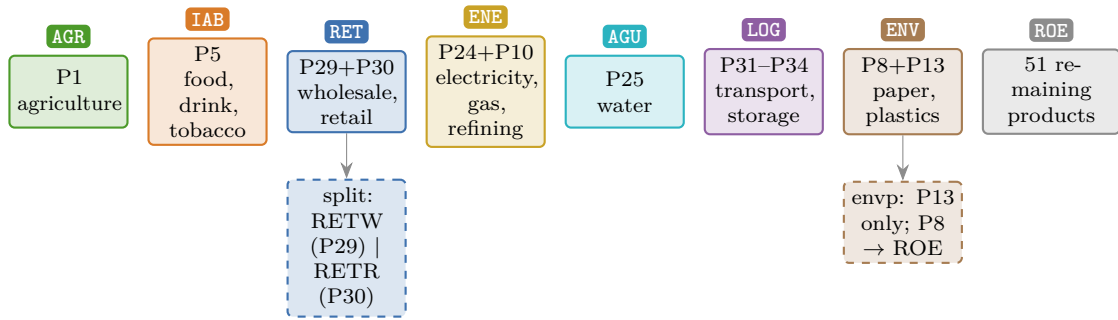


(b) The reference time T_i fixes an isolated exponential recovery $q_j(t) = q_j(0) e^{-t \ln 100 / T_j}$ that reaches 1 % at T_j . Heat multiplies T of AGR by 1.5 and of AGU by 2.5.

Figure 4: The two ingredients of the model: the interdependence matrix calibrated on INE data and the recovery convention declared per scenario.

4.1 From 65 INE products to 8 sectors

`ine_calibration.py` aggregates the 65 CPA products of the INE symmetric table by summing flows ($Z_{\text{agg}}[I, J] = \sum_{p \in I, q \in J} Z[p, q]$, $x_{\text{agg}}[I] = \sum_{p \in I} x[p]$). The base partition is shown in Figure 5; two alternative partitions serve the aggregation-sensitivity analysis.



dashed: alternative partitions written to `ine_calibration_split.json` and `ine_calibration_envp.json`

Figure 5: Base partition of the INE products into eight sectors and the two variants used by `partition_sensitivity.py`. Documented ambiguities: fishing and forestry outside AGR, vehicle trade outside RET, postal services outside LOG, tobacco inside IAB, ENV a partial packaging proxy.

4.2 Scenario catalogue

All scenarios are declared once in `scenarios.py` (dictionaries `SCN`, `RANGES`, `A1_TWO_PHASE`). Point values are the modes of triangular exploratory distributions; the brackets give their ranges. Baseline reference recovery times are $\tau = (20, 10, 7, 4, 8, 6, 6, 10)$ days for (AGR, IAB, RET, ENE, AGU, LOG, ENV, ROE).

Code	Scenario	Target	Heat	$q(0)$ point [range]	T days [range]
S1	Single food-industry operator at a standstill	IAB		0.02 [0.005, 0.03]	5 [3, 10]
S2	Coordinated industry campaign (harvest, sector-wide stress)	IAB		0.20 [0.10, 0.25]	14 [7, 45]
S3	Irrigation SCADA/IoT compromise	AGR		0.15 [0.10, 0.20]	10 [7, 14]
S4	Irrigation SCADA/IoT compromise under heatwave	AGR	☼	0.15 [0.10, 0.20]	25 [15, 35]
S5	Compound: industry + irrigation, heatwave	IAB AGR	☼	0.20 / 0.15	14 / 25
A1	Silent fertigation integrity drift, two-phase	AGR		$\bar{c} = 0.10$ [0.05, 0.15]	$t_d = 30$ [21, 45]; $T_{\text{rem}} = 10$ [7, 14]
A1-T	Transient comparison case of A1	AGR		0.10 [0.05, 0.15]	30 [21, 45]
G1	Livestock climate/ventilation control	AGR		0.12 [0.08, 0.15]	8 [5, 12]
R1	Retail ransomware: ERP, distribution centres, cold chain	RET		0.04 [0.01, 0.05]	9 [5, 45]
C1	Mediterranean-arc compound (harvest + heat)	AGR IAB RET	☼	0.15 / 0.20 / 0.04	25 / 14 / 9

Table 1: Scenario set as declared in `scenarios.py`. ☼ marks heat-conditioned recovery (AGU $\times 2.5$, AGR $\times 1.5$; Monte Carlo ranges $U[1.5, 3.5]$ and $U[1.2, 2.0]$). A1 is the principal two-phase representation (sustained depth until detection, then remediation); A1-T is its transient counterpart.

5 Script by script

 `ine_calibration.py -partition base|split|envp`

stages 1–3

Reads `data/cne_tio_22.xlsx`. Writes `ine_calibration.json` (or `_split`, `_envp`), `ine_Astar_8x8.csv`, `ine_x_8.csv`.

Reads sheet `Tabla2` (domestic product-by-product flows, EUR million), the row *Producción a precios básicos* of `Tabla1` (output vector) and `Tabla5` (technical coefficients, used only as an independent check of $A = Z \text{diag}(x)^{-1}$). Aggregates to the partition of Figure 5, checks the accounting identities and the spectral radius $\rho(A^*) < 1$, and stores x , A , A^* , the backward and forward exposure weights and the output multipliers.

 `ine_trade_margins.py`

stage 4

Reads `data/cne_tod_22.xlsx`. Writes `ine_trade_margins.json`.

From the supply table at basic prices derives the output of trade services (products 64 and 65, which equals x_{RET}), the net trade margins reassigned by INE, the margins generated on agri-food products and the output of meat products. With the 2022 turnover of wholesale and retail trade (INE structural business statistics) it converts the net sales of the largest grocer into a margin-basis output and a share of RET of 0.040–0.042, which fixes $q_{\text{RET}}(0) = 0.04$ in scenario R1.



ine_scenarios.py

stage 5

Reads the calibration JSON and scenarios.py. Writes ine_scenarios_results.json and mc_samples.npz.

Computes the point losses $\Gamma(120)$ and Γ_∞ of every scenario, the dual (backward/forward) ranking for S2, and the exposure weights. Runs the exploratory Monte Carlo: $N = 5000$, seed 20260719; non-zero $a_{ij}^* \times U[0.8, 1.2]$, $x_i \times U[0.9, 1.1]$, triangular q and T , heat multipliers as in Table 1. A Gaussian-copula sweep with equicorrelation $\rho \in \{0, 0.5, 1\}$ among the q and T of a scenario is reported; rank frequencies are recorded under three prioritisation criteria. The structural draws (A^* , $\hat{x}$, heat multipliers) are saved to mc_samples.npz so that later stages reuse exactly the reference run. Slow stage: about 14 s.



diim.py

stage 6, verification

Reads ine_scenarios_results.json and scenarios.py. Writes nothing. Exit code 1 on any mismatch.

Independent checks of the implementation: matrix-exponential trajectories against an ODE integrator; the loss identity $\Gamma(H) = x^\top W(H)/365$; the closed form $\Gamma_\infty = x^\top (I - A^*)^{-1} K^{-1} q(0)/365$; linearity in T ($\Gamma_\infty(S4)/\Gamma_\infty(S3) = 2.5$); additivity $C1 = S4 + S2 + R1$ at infinite horizon; forward invariance of $[0, 1]^n$; and equality of every point loss with the stored baseline. Because the stage fails loudly, a stale or edited reproduction set cannot pass silently.



static_vs_dynamic.py

stage 7

Writes criteria.json.

Three prioritisation criteria among the focal sectors, each answering a different question: w_j (equal integrated inoperability), $m_j = w_j/x_j$ (equal integrated monetary withdrawal; the Leontief output multiplier) and $v_j = \tau_j(1 - a_{jj}^*)w_j/(365 \ln 100)$ (loss per unit of initial inoperability at the baseline recovery time).



partition_sensitivity.py

stage 8

Reads the three calibration JSON files. Writes partition_sensitivity.json.

Recomputes criteria and the losses of S2, S4 and C1 under the **base**, **split** and **envp** calibrations. Under **split**, R1 is re-specified on retail trade (RETR) with a point injection of 0.10 (range 0.05–0.12).



recovery_shape.py

stage 9

Writes recovery_shape.json.

Compares recovery paths with the same $q_j(0)$: exponential (reference), plateau then exponential, plateau with the same end date, linear ramp and three steps. With the target's path exogenous, Γ_∞ scales exactly with the area under that path; the DIIM proper (endogenous target) is also reported. A numerical integration checks the closed form.



uncertainty_decomposition.py

stage 10

Reads mc_samples.npz. Writes uncertainty_decomposition.json.

(a) Envelopes under uniform and PERT marginals instead of triangular, reusing the stored structural draws. (b) One-at-a-time variance shares on the log scale: depth, recovery, structure. (c) Convergence: split-half percentiles and three further seeds. Slowest stage: about 32 s.

 <code>decision_example.py</code>	stage 11
Reads <code>mc_samples.npz</code> . Writes <code>decision_example.json</code> .	
Illustrative protection decision on S2 with two <i>assumed</i> measures: depth control ($q_{IAB}(0)$ from 0.20 to 0.18) and restoration control (T from 14 to 12 days). Reports the avoided loss of each, the joint benefit and the cross term, with envelopes from the stored draws.	
 <code>external_contrast.py</code>	stage 12
Writes <code>external_contrast.json</code> .	
Converts two documented incidents (JBS 2021; Maersk/NotPetya 2017 as compiled by Welburn and Strong) into full-stop-equivalent days $D_{eq} = T / \ln 100$ and contrasts the triangular T ranges with the Sophos 2025 recovery-time survey.	
 <code>agri_bridge.py</code>	stage 13
Writes <code>agri_bridge.json</code> .	
Decomposes the agricultural injection as $q_{AGR}(0) = \pi \varphi h$ with public anchors for π (MAPA 2024 crop, irrigation, pigmeat and poultry shares). It does not estimate φ or h ; the products implied by the point injections and ranges are printed and flagged as pending operator data.	
 <code>extensions_L2_L7.py</code>	stage 14
Writes <code>extensions.json</code> .	
L2: two-phase integrity attack A1 over detection latency and remediation time, and its ratio to the transient A1-T. L3: adversarial timing premium with a seasonal T_{AGR} . L4: single-counted channel corridor $A_{max} = \max(A^*, A_{fwd})$ element-wise, with Γ_∞ and the focal ranking under both matrices.	
 <code>regional_reading.py</code>	stage 15
Writes <code>regional_reading.json</code> .	
National loss of an incident confined to Andalusia or the Mediterranean arc (INE 2024 output shares as proxies); location of Γ_∞ by receiving sector in S2, S4, R1, C1; sensitivity of $\Gamma_\infty(S4)$ to AGR's purchase and sales structure.	
 <code>make_figures.py</code>	stage 16
Reads <code>ine_scenarios_results.json</code> , <code>criteria.json</code> , <code>partition_sensitivity.json</code> , <code>recovery_shape.json</code> , <code>uncertainty_decomposition.json</code> . Writes <code>../figs/*.png</code> .	
Draws the six manuscript figures at 300 dpi (single column 3.4 in, double column 7.0 in). No number is typed in the script. The output directory is created if missing.	

6 What a correct run looks like

A faithful run ends with the line PIPELINE COMPLETED in 50 s (give or take the speed of the machine) and leaves the JSON, CSV and NPZ files identical to the committed ones. The values of Table 2 are read directly from those files and identify the reference run.

Quantity	Value	File	Key
$\rho(A^*)$	0.4130	 <code>ine_calibration.json</code>	<code>rho</code>
Output x (EUR M): AGR, IAB, RET	61 984; 160 266; 245 096	 <code>ine_calibration.json</code>	<code>x</code>
Exposure weights w (EUR M): RET, IAB, LOG	405 025; 362 960; 291 364	 <code>ine_calibration.json</code>	<code>w_backward</code>
$\Gamma(120)$ of S1, S3, G1, R1 (EUR M)	17.9; 104.6; 67.0; 84.2	 <code>ine_scenarios_results.json</code>	<code>baseline</code>
$\Gamma(120)$ of S2, S4, S5, C1 (EUR M)	501.6; 261.6; 763.2; 847.4	 <code>ine_scenarios_results.json</code>	<code>baseline</code>
$\Gamma(120)$ of A1-T and two-phase A1 (EUR M)	209.3; 912.0	 <code>ine_scenarios_results.json</code>	<code>baseline</code>
Cyber-climate margin S4 – S3 (EUR M)	157.0	 <code>ine_scenarios_results.json</code>	<code>marginal_climate</code>
S2 envelope P5 / P50 / P95, $\rho = 0$ (EUR M)	324 / 668 / 1292	 <code>ine_scenarios_results.json</code>	<code>envelopes</code>
Monte Carlo size, valid draws, seed	5000, 5000, 20260719	 <code>ine_scenarios_results.json</code>	<code>n_samples, n_valid, seed</code>
Largest grocer's share of RET	0.0401–0.0420	 <code>ine_trade_margins.json</code>	<code>largest_grocer</code>

Table 2: Values that identify a faithful run. All were read from the committed files on 3 October 2026 after regenerating them.

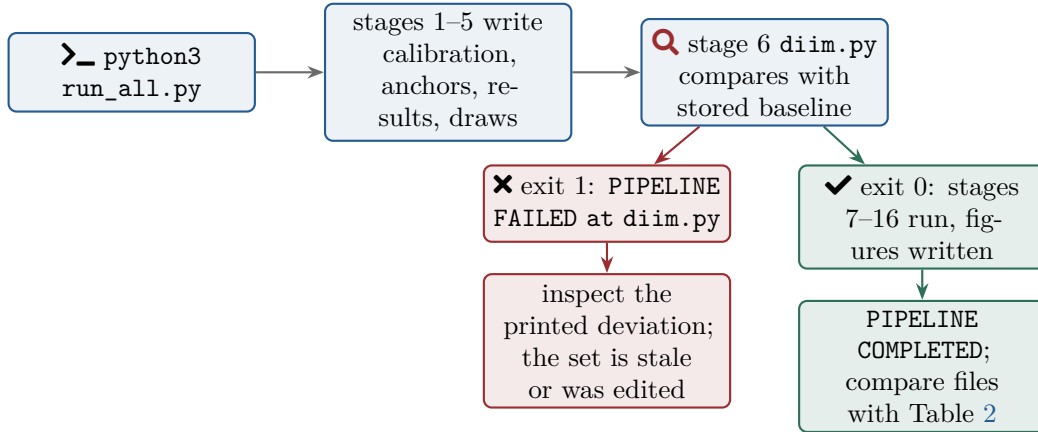


Figure 6: Decision path of a run. The verification stage sits between the result-producing stages and the analysis stages, so downstream JSON files are only rewritten from a baseline that passed.

✔ Verification performed for this guide

On 3 October 2026 the directory was copied to a clean location with the pinned package versions and `run_all.py` was executed. The run completed in 50 s. Every JSON and CSV file and `mc_samples.npz` were compared with the committed copies with `cmp`: no differences. The six PNG figures were written to `../figs/`. The offline Censys tests (`python3 -m unittest test_censys_exposure`) passed, 5 tests.

7 Data provenance

File	Source	Read by
data/ cne_tio_22.xlsx	INE, <i>Contabilidad Nacional Anual de España, Revisión Estadística 2024</i> , symmetric input–output tables, reference year 2022, published 19 December 2025. Sheets Tabla1 (production at basic prices), Tabla2 (domestic product-by-product flows, 65 products, EUR M), Tabla5 (technical coefficients, check only).	 ine_calibration.py
data/ cne_tod_22.xlsx	INE, same release, supply and use tables 2022. Sheet Tabla1 (supply table at basic prices with the transformation to purchasers’ prices).	 ine_trade_margins.py

Table 3: Input data. “Revisión Estadística 2024” names the accounting revision; the reference year of both tables is 2022. The workbooks are official public data under INE’s conditions of use and are never modified by the scripts.

Other anchors (MAPA agricultural accounts, Mercasa, INE structural business statistics, the largest grocer’s net sales, incident reports and the Sophos survey) are typed as constants in the scripts that use them, with their sources in the docstrings and in the manuscript’s reference list.

Scenario-conditioned, not predictive

The calibration is empirical. The shock depths, recovery times and climate multipliers are declared scenario assumptions, and the Monte Carlo percentiles are exploratory simulation envelopes, not confidence intervals. Nothing produced by this set is a forecast.

8 The Censys exposure material

The manuscript has a supplement on the Internet-facing exposure, in Spain, of the device classes attacked in the scenarios, built from *aggregate* Censys Platform counts (no host lists, no individual records). The files in `censys/` and the three `censys_*` scripts belong to that supplement. None of them is part of `run_all.py`, and two of the scripts need inputs that are deliberately not shipped (Figure 7).

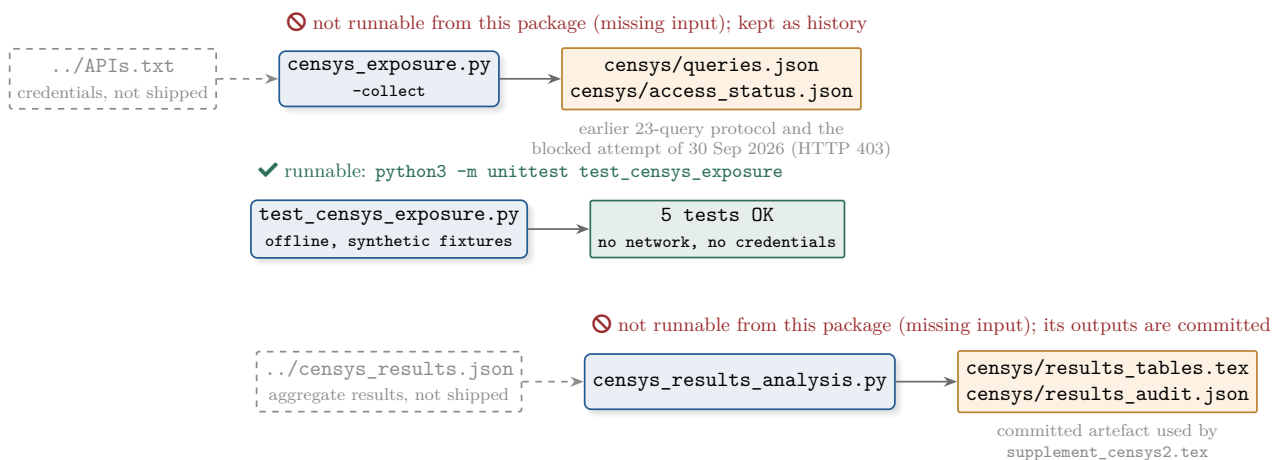


Figure 7: The Censys material. Dashed grey boxes are inputs that are not distributed. Only the offline test module can be executed from a clean copy.

- `censys_exposure.py` implements the earlier aggregate-only query protocol. Its default action is offline plan generation; `-collect` would issue 23 aggregate requests to the documented endpoint and consumes account credits. It reads credentials from `../APIs.txt`, which is not part of the package. `censys/access_status.json` records that the attempt of 30 September 2026 was blocked before any data was retrieved.
- `censys_results_analysis.py` audits the aggregate results supplied in `../censys_results.json` (37 queries, SHA-256 recorded in the audit) and writes the LaTeX macros of `censys/results_tables.tex` and the checks of `censys/results_audit.json`. Standard library only. The source file is not shipped, so the committed outputs are the reproduction artefact.
- `censys/README.md` (in Spanish) states the scope, units and limits of that analysis: the port-based counts are not host or device censuses, no agri-food attribution is applied, and exposure is not mapped onto any DIIM parameter.

9 The six figures

Stage 16 writes the figures of Figure 8 to `../figs/`. Each panel is drawn from the JSON files alone; sector colours are fixed across figures and one hue is used per measure.

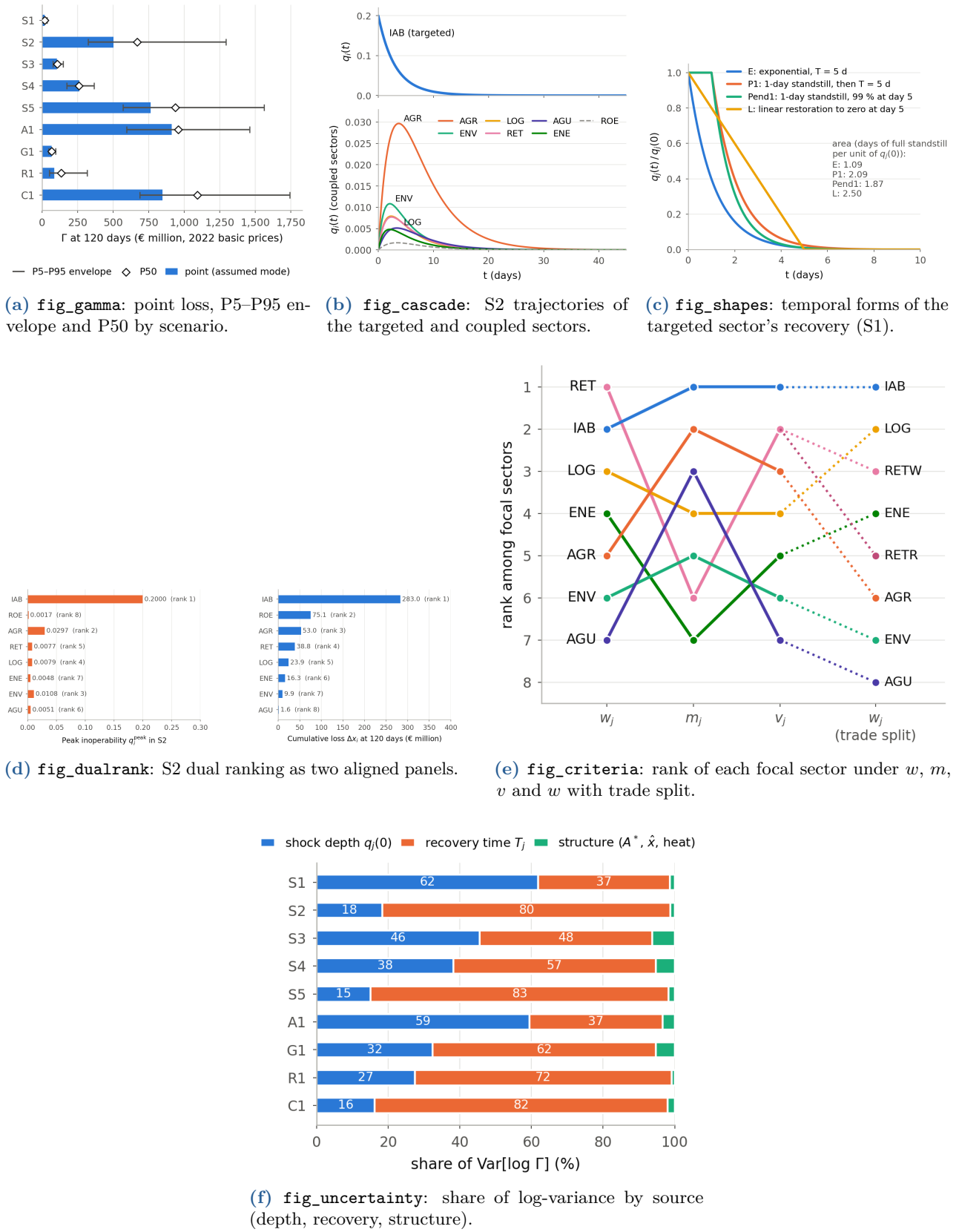


Figure 8: The manuscript figures as regenerated by `make_figures.py` on 3 October 2026.

10 Before editing anything

- **Change scenarios only in `scenarios.py`.** Every downstream script imports its dictionaries; assertions there check that each point value lies inside its range.
- **Rerun from stage 5 after any change upstream.** The verification stage compares against the stored `ine_scenarios_results.json`; if you change the calibration or a scenario, `ine_scenarios.py` must rewrite that file first, otherwise `diim.py` will stop the pipeline by design.
- **Do not delete `mc_samples.npz` casually.** Stages 10 and 11 reuse these structural draws; `ine_scenarios.py` regenerates the file, but only with the pinned numpy stream will it be bit-identical.
- **Figures go outside the clone.** `make_figures.py` writes to the sibling directory `../figs/`, which the manuscript includes.
- `__pycache__/` and `run_scenarios.log` are local artefacts and can be removed.